

HealMesh — Constitution

Status: Draft v1 Authority: This is the canonical source for the project's non-negotiable principles. `AGENTS.md` restates the subset relevant to AI coding agents; `docs/TDD.md` and `docs/TESTING.md` implement and verify these principles at the component level. If any document conflicts with this one, this one wins, and the conflicting document should be corrected.

This file changes rarely and deliberately. Amending it requires the same rigor as amending any other constitution: a recorded reason, not a quiet edit under deadline pressure. If a task seems to require violating a principle here, that is a signal to reconsider the task, not the principle — and if the principle genuinely needs to change, that change itself belongs in `DECISION_LOG.md` with full reasoning, not a silent diff to this file.

Article 1 — Purpose

HealMesh exists to shorten the time between "something in the cluster is broken" and "a human understands why," and, later, to safely close a narrow set of well-understood fixes without waiting on a person — never to replace human judgment on anything ambiguous or high-blast-radius. Diagnosis is the product. Remediation is an earned, gated extension of it, not a separate goal pursued in parallel.

Article 2 — The seven invariants

These are restated verbatim from the TDD and AGENTS.md because they are the actual constitution of the system — everything else is negotiable engineering detail; these are not.

1. **The LLM never produces an executed command.** Output is always forced into a closed, typed enum before anything downstream sees it. Unparseable output means no automated action, never a best guess.
2. **No orchestration middleman between the core logic and the LLM API.** Direct SDK calls only, so the security-critical path is code the team fully owns and can audit line by line.
3. **Read and write are architecturally separate,** with separate code paths and separate identities, even when co-located for deployment convenience.
4. **Secure by default, opt-in to widen.** Every new namespace, action type, or integration starts at the narrowest permission and is widened only by explicit configuration.
5. **The audit trail is append-only at the code level,** not just the database level — no

delete or update method exists for audit records, full stop.

6. **The hardcoded namespace denylist is enforced in code**, inside the executor itself, and cannot be overridden by any configuration, policy, or LLM output.
 7. **Anything beyond the single lowest-blast-radius action (`SCALE`) always requires human approval**, permanently — this is a standing design decision, not a v1 limitation awaiting removal.
-

Article 3 — Sequencing discipline

The project is built in the order that lets each layer earn trust before the layer above it depends on that trust:

- Diagnosis before remediation. A system that explains failures well but does nothing is a complete, useful product on its own (per the PRD) — it is not a stepping stone to be rushed past.
- A benchmark before scaling scope. Accuracy claims are backed by a measured, honestly reported benchmark before any execution capability ships (Phase 1.5 gate).
- One action type before many. `SCALE` is proven, gated, and stable before `PATCH`, `REDEPLOY`, or `HELM_UPGRADE` are added, one at a time.
- Breadth (Dashboard, SDK, multi-cluster, enterprise features) only once there is a real, named driver for it — never built speculatively ahead of demand. Premature breadth is a named risk in the PRD, not a hypothetical one.

Deadline pressure is not a valid reason to skip a phase gate. If a deadline genuinely requires it, that tradeoff is made explicitly and recorded in `DECISION_LOG.md`, not made implicitly by quietly shipping ahead of the gate.

Article 4 — Honesty as a design principle

- Accuracy claims (diagnosis benchmark results, false-positive rates) are reported exactly as measured, including when they fall short of target. This is a functional requirement in the PRD, not an aspiration.
 - The system is never described — internally or externally — as "fully autonomous" or framed in a way that understates the human-approval requirement that Article 2.7 guarantees.
 - Weak spots (a failure type with lower diagnosis accuracy, a component with lower test coverage than its bar in `TESTING.md`) are surfaced, not smoothed over in status updates.
-

Article 5 — Who can change what

- **Article 2 (the seven invariants)** can only be changed by a decision recorded in `DECISION_LOG.md` with explicit reasoning, reviewed by more than one person on the team. An AI coding agent must never modify this article or act as if it has been changed without seeing that recorded decision.
 - **Article 3 (sequencing)** can be adjusted per real project constraints (e.g., a legitimate reason to reorder Phase 3 sub-tasks), but the *existence* of phase gates themselves — especially the Phase 1.5 benchmark gate — follows the same bar as Article 2.
 - **Everything else in this document** (Articles 1 and 4) reflects intent and can be clarified through normal editing, but should still be discussed rather than silently rewritten, since they set the tone for every other document in this repository.
-

Article 6 — Relationship to other documents

Document	Role relative to this Constitution
<code>AGENTS.md</code>	Operationalizes Article 2 for AI coding agents specifically
<code>docs/TDD.md</code>	Implements Article 2 and Article 3 at the component level
<code>docs/PRD.md</code>	Implements Article 1 and Article 3 as product scope and phase gates
<code>docs/TESTING.md</code>	Verifies Article 2 is actually true in the shipped system
<code>DECISION_LOG.md</code>	The record of any change to, or exception considered against, this document
<code>MIGRATION.md</code>	Documents how the prior prototype's design (which violated Article 2's "no orchestration middleman" principle) is being retired in favor of this Constitution

If a new document is added to this repository, its author should note here how it relates to this Constitution, even briefly.